PWA Web para Controle Descentralizado de Estoque FIFO

Documento de Regras de Negócio, Arquitetura e Plano de Testes
Projeto: PWA Web para Controle Descentralizado de Estoque FIFO
Base Técnica: README do GitHub, Diagramas UML, Modelagem ER, Dump MySQL/MariaDB, Backend Java (JDBC) e Frontend React.

1. Convenções e Legenda de Status

Para garantir clareza e padronização visual, os status e camadas do sistema são identificados pelas seguintes legendas:

- **Legenda de Status:**
  - **OK:** Funcionalidade implementada e validada.
  - **PARCIAL:** Funcionalidade existente com falhas ou restrita a apenas uma camada.
  - **FALTA:** Funcionalidade ainda não desenvolvida.
  - **PROPOSTA:** Sugestão de melhoria ou regra a ser confirmada pela equipe.
- **Legenda de Camadas:**
  - **B:** Banco de Dados (PostgreSQL / MySQL / MariaDB).
  - **S:** Backend / Service (Java / JDBC).
  - **F:** Frontend / PWA (React).

2. Regras de Negócio Fundamentais

Regra 1: Impedir Venda/Saída de Produto com Saldo Insuficiente

O sistema deve proibir rigorosamente movimentações do tipo SAÍDA caso o saldo do produto seja nulo ou inferior à quantidade solicitada. Esta validação atua em quatro camadas de segurança contínuas:

Camada	Mecanismo de Validação	Status
Front-end (React)	Prévia do pedido valida saldo e indica "Saldo insuficiente: faltam X", bloqueando a gravação.	**OK** (api.saida valida faltam > 0 antes de enviar).
Regra Pura (RegraFifo)	Valida os códigos de erro: SEM_ESTOQUE, LOTE_SEM_SALDO, SALDO_INSUFICIENTE.	**OK** (Testado unitariamente).
Backend (EstoqueService)	Dispara exceções de saldo,	**PARCIAL** (Contém falha de

Camada	Mecanismo de Validação	Status
	mas necessita de ajuste transacional (ver Seção 4).	atomicidade).
Banco de Dados	Constraint CHECK (quantidade >= 0) em saldo_lote_local impede saldos negativos.	**OK**

Regra 2: Respeito Rigoroso à Fila FIFO (First-In, First-Out)

A saída de produtos deve priorizar compulsoriamente os lotes com datas de entrada mais antigas. Abaixo constam as opções de implementação do comportamento do sistema:

Opção	Comportamento	Coerência e Aplicação
A. Bloquear Sempre	Qualquer tentativa de baixa em lote mais novo havendo lote antigo disponível é sumariamente recusada.	Rígido; não atende imprevistos reais (lote avariado, solicitação específica).
B. Bloquear com Exceção Justificada (Recomendada)	Recusa a saída normal; permite apenas se o usuário fornecer uma justificativa formal, gravando fifo_ignorado = true.	**Recomendada:** Já suportada na modelagem e no método RegraFifo.validarBaixaLote.
C. Permitir Livremente	Qualquer lote pode ser baixado em qualquer ordem sem restrições.	Incompatível com os requisitos de controle FIFO do projeto.

3. Catálogo Completo de Regras de Negócio (RN)

3.1 Gestão de Produtos

ID	Regra de Negócio	Camada	Código / Retorno	Status
RN-P01	SKU deve ser obrigatório e único.	B, F	duplicidade (1062)	**OK** (B, F); Backend não valida.

ID	Regra de Negócio	Camada	Código / Retorno	Status
RN-P02	Nome e unidade de medida são obrigatórios.	F, S	ENTRADA_INVALIDA	**PARCIAL** (Apenas F; B aceita texto vazio).
RN-P03	Estoque mínimo deve ser >= 0.	F, B	-	**PARCIAL** (F OK; B sem CHECK).
RN-P04	Inativação lógica do produto (nunca exclusão física para preservar histórico).	S, F	-	**OK**
RN-P05	Produtos inativos não podem receber novas ENTRADAS.	S, F	-	**OK**
RN-P06	Produtos inativos permitem SAÍDA e TRANSFERÊNCIA (para zerar saldo).	S	-	**OK**
RN-P07	Operação com produto inexistente deve retornar erro claro.	S	-	**PARCIAL** (Validado apenas na entrada).

3.2 Locais de Estoque (Descentralizado)

ID	Regra de Negócio	Camada	Status
RN-L01	Nome do local de estoque deve ser único.	B	**OK**

ID	Regra de Negócio	Camada	Status
RN-L02	O local informado na operação deve ser existente e válido.	S, F	**OK**
RN-L03	Saldos são independentes por local; saídas consomem apenas lotes do local selecionado.	S, F	**OK**
RN-L04	Local inativo não pode receber entrada nem ser destino de transferência.	S, F	**PROPOSTA**

3.3 Gestão de Lotes

ID	Regra de Negócio	Camada	Status
RN-LT01	Toda entrada de mercadoria cria um novo lote.	S, F	**OK**
RN-LT02	Código do lote é obrigatório (não nulo e não em branco).	S, F	**PARCIAL** (Validação falta no Service).
RN-LT03	Código do lote deve ser único dentro do mesmo produto.	B, F	**PARCIAL** (Presente no Dump SQL, ausente no txt).
RN-LT04	Data de entrada define a ordem FIFO e é gerada automaticamente pelo servidor.	S	**OK** (Garantido pelo LoteDAO).
RN-LT05	Data de validade do lote não pode ser anterior à data de entrada.	S, F	**PROPOSTA**

ID	Regra de Negócio	Camada	Status
RN-LT06	Quantidade inicial do lote deve ser estritamente maior que 0.	S, B	**PARCIAL** (S OK; B permite 0).
RN-LT07	Saldo de um lote por local nunca pode ser negativo.	B, S	**OK**
RN-LT08	A soma dos saldos de um lote entre todos os locais não pode exceder sua quantidade inicial.	B	**PROPOSTA** (Auditado via Query A2).

3.4 Saída / Venda

ID	Regra de Negócio	Camada	Código de Erro	Status
RN-S01	Quantidade > 0, com até 3 casas decimais.	S, F	QUANTIDADE_INVALIDA	**OK** na RegraPura; Service valida apenas > 0.
RN-S02	Impedir venda com saldo 0 (Regra 1).	F, S, B	SEM_ESTOQUE, LOTE_SEM_SALDO	**OK** no Front e RegraPura; **FALHA** no Service.
RN-S03	Operação atômica "Tudo ou Nada": não baixar parcialmente se não houver saldo total.	F, S	SALDO_INSUFICIENTE	**OK** no Front; **FALHA** no Service (Ajuste Seção 4).
RN-S04	Saída automática consome lote mais antigo; empate desempata por	S, F, B	-	**OK**

ID	Regra de Negócio	Camada	Código de Erro	Status
	ID.			
RN-S05	Furar a fila FIFO gera erro e exige exceção tratada.	S	FIFO_VIOLADO	**OK** na classe RegraFifo.
RN-S06	Exceção ao FIFO requer justificativa obrigatória e grava fifo_ignorado = true.	S, B	FIFO_VIOLADO	**OK** na classe RegraFifo.
RN-S07	Apenas perfis GESTOR ou ADMIN podem aprovar exceção FIFO.	S	-	**PROPOSTA**
RN-S08	Saídas abrangendo múltiplos lotes geram um registro de movimentação por lote.	S	-	**OK**
RN-S09	Lote vencido exibe alerta antes de permitir consumo.	S	-	**OK** (Emite alerta).
RN-S10	Notificar alerta quando saldo total do produto atingir o estoque mínimo.	S, F	-	**PROPOSTA**

3.5 Entrada e Transferências

ID	Regra de Negócio	Camada	Status
RN-E01 / E02	Entrada adiciona saldo ao local de destino e gera movimentação do tipo ENTRADA.	S, F, B	**OK**
RN-E03	Novas entradas ingressam no final da fila FIFO (data_entrada = AGORA).	S	**OK**
RN-E04	Operação de entrada é atômica (cria lote, saldo e movimentação juntos ou nada).	S	**PARCIAL** (Depende da correção da transação).
RN-T01 / T02	Transferência exige locais de origem e destino válidos e distintos.	S, F	**OK**
RN-T03 / T04	Transferência consome origem via FIFO e preserva o lote/idade no destino.	S, F	**OK**

3.6 Movimentação, Histórico e PWA Offline

ID	Regra de Negócio	Camada	Status
RN-M01 a M03	Toda operação registra movimentação contendo usuário, data/hora e quantidade.	B, S	**OK**
RN-M04 / O02	Idempotência via uuid_cliente: requisições duplicadas não	B, F	**PARCIAL** (B OK; Java precisa reutilizar o UUID enviado pelo PWA).

ID	Regra de Negócio	Camada	Status
	geram múltiplas baixas.		
RN-M05	Histórico de movimentações é imutável e inalterável.	B	**PARCIAL** (Proposta P4: Triggers de bloqueio).
RN-O01	Operações offline são salvas e sincronizadas automaticamente no retorno da conexão.	F	**OK** (Atributo sincronizado).
RN-O03	Sincronização revalida FIFO e saldo; conflitos em saldos zerados disparam recusa formal.	S	**FALTA**

4. Diagnóstico Técnico e Correção da Transacionalidade

Problema Crítico Identificado: Na implementação atual de EstoqueService, métodos como registrarSaida executam c.setAutoCommit(false). Contudo, os DAOs internos obtêm novas conexões via Conexao.getConexao() com autocommit ativado. Como consequência, cada UPDATE e INSERT é efetivado imediatamente no banco. Em cenários de saldo insuficiente no meio da execução, a exceção é lançada mas o rollback() não reverte as alterações parciais já confirmadas.

Solução Recomendada:

1. Injetar a instância de Connection em todos os métodos do DAO.
2. Planejar a operação previamente com a classe RegraFifo.planejarSaida antes de efetuar qualquer alteração no banco de dados.

```
Connection c = Conexao.getConexao();
try {
   c.setAutoCommit(false);

   // Busca saldos aplicando FOR UPDATE para concorrencia
   List<LoteSaldo> saldos = saldoDAO.listarSaldos(c, produtoId, localId);

   // Valida regras puras FIFO em memoria
   PlanoSaida plano = RegraFifo.planejarSaida(saldos, qtd, LocalDate.now());
```

```
    // Efetiva as movimentacoes na mesma conexao
    for (ItemPlano i : plano.getItens()) {
        saldoDAO.somar(c, i.getLoteId(), localId, i.getQuantidade().negate());
        movimentacaoDAO.registrar(c, nova(TipoMovimentacao.SAIDA, i.getLoteId(), localId,
null, i.getQuantidade(), usuarioId));
    }

    c.commit();
} catch (SQLException | RuntimeException e) {
    c.rollback();
    throw e;
} finally {
    c.setAutoCommit(true);
    c.close();
}
```

5. Plano de Testes

5.1 Testes de Integração Backend + Banco de Dados

ID	Cenário do Teste	Resultado Esperado	Regra Associada
IT-01	Saída de 15 unidades de um produto.	Saldos atualizados conforme FIFO; 2 movimentações geradas.	RN-S04, RN-S08
IT-02	Saída de 45 (zerando estoque) seguida de nova saída de 1.	Primeira saída sucede; segunda é recusada com SEM_ESTOQUE.	RN-S02
IT-03	Saída de 100 em estoque com apenas 45 disponíveis.	Retorna erro SALDO_INSUFICIENTE; nenhum saldo é alterado e zero registros são gravados.	RN-S03 (Atomicidade)
IT-04	Saída da quantidade exata do saldo disponível (45).	Todos os lotes zeram; novas tentativas são bloqueadas.	RN-S02, RN-S03

ID	Cenário do Teste	Resultado Esperado	Regra Associada
IT-05	Saída com quantidade zero, negativa ou nula.	Erro QUANTIDADE_INVALIDA; nenhuma gravação efetuada.	RN-S01
IT-06	Operação com IDs de produto ou local inexistentes.	Retorna erro de validação sem alterar dados.	RN-P07, RN-L02
IT-07	Saída em local sem estoque, havendo saldo em outro local.	Recusada com SEM_ESTOQUE no local solicitado; outro local permanece intacto.	RN-L03
IT-08	Entrada de novo lote (L004) com 30 unidades.	Lote criado, saldo atualizado e posicionado no final da fila FIFO.	RN-E02, RN-E03
IT-09	Tentativa de entrada em produto inativado.	Operação bloqueada; nenhum lote criado.	RN-P05
IT-10	Baixa manual fora da ordem FIFO sem justificativa.	Recusada com erro FIFO_VIOLADO. Com justificativa, é aprovada e marca fifo_ignorado = true.	RN-S05, RN-S06
IT-11	Concorrência: duas solicitações simultâneas de saída de 40 (saldo 45).	Apenas uma transação é confirmada; a outra é recusada por saldo. Saldo final = 5.	RN-S03, Concorrência

6. Roadmap de Implementação Sugerido

1. **Ajuste de Transacionalidade e Conexões:** Corrigir a gestão de conexões JDBC em EstoqueService garantindo commit/rollback atômico.
2. **Execução da Suíte de Testes IT-01 a IT-16:** Executar e validar todos os cenários de teste de integração.
3. **Harmonização de Schemas SQL:** Sincronizar o arquivo BancoDeDados.txt com o dump do banco, adicionando constraints e triggers necessárias.

4. **Módulo de Seleção Manual de Lote:** Desenvolver a interface para escolha de lote, ativando as regras de exceção FIFO com justificativa.
5. **Segurança e Autenticação:** Implementar senhas hash, autenticação JWT e controle de perfis de usuário (OPERADOR, GESTOR, ADMIN).
6. **Sincronização PWA Offline:** Tratar reaplicação de regras e resolução de conflitos para dados offline.